Lecture 10: Parallel and Scalable Data Processing

Central Issue: Large data file does not fit entirely in DRAM

Basic Idea: Divide-and-conquer again. "Split" a data file (virtually or physically) and *stage reads* of its pages from disk to DRAM; vice versa for writes

❖ 4 key regimes of scalability / staging reads:

- **Single-node disk**: Paged access from file on local disk
- **Remote read**: Paged access from disk(s) over a network
- **Distributed memory:** Data fits on a cluster's total DRAM
- **Distributed disk**: Use entire memory hierarchy of cluster

Page Management in DRAM Cache

- **Caching**: Retaining pages read from disk in DRAM
- **Eviction**: Removing a page frame's content in DRAM
- **Spilling**: Writing out pages from DRAM to disk

❖ If a page in DRAM is "**dirty**" (modified but not yet written back to disk; for example, if some bytes have been written to page), then eviction requires a spill; o/w, ignore that page

❖ The set of DRAM-resident pages typically changes over the lifetime of a process

❖ **Cache Replacement Policy**: The **algorithm** that chooses which page frame(s) to evict when a new page has to be cached but the OS cache in DRAM is full

Quantifying I/O: Disk, Network

- Page reads/writes to/from DRAM from/to disk incur latency
- **Disk I/O Cost**: Abstract counting of number of page I/Os; can map to bytes given page size
- Sometimes, programs read/write data over network
- **Communication/Network I/O Cost**: Abstract counting of number of pages/bytes sent/received over network
- *Note*: I/O cost is *abstract*; mapping to latency is *hardware-specific*

Example: Suppose a data file is 40GB; page size is 4KB
I/O cost to read file = 10 million page I/Os
Disk with I/O throughput: 800 MB/s -> 40GB / (800MB/s) = 50s
Network with speed: 200 MB/s -> 40GB / (200MB/s) = 200s

Scaling to (Local) Disk

❖ In general, *scalable programs stage* access to pages of file on disk and efficiently use available DRAM

- Recall that typically DRAM size << Disk size

❖ Modern DRAM sizes can be tens of GBs; so we read a "chunk"/"block" of file at a time (say, 1000s of pages)

- On magnetic hard disks, such chunking leads to *more sequential* I/Os, raising throughput and lowering latency!
- Similarly, for any eviction event needed, there is also a write of chunk of dirtied pages at a time.

Generic Cache Replacement Policies

***Q:** What to do if number of page frames is too few for file?*

❖ **Cache Replacement Policy**: Algorithm to decide which page frame(s) to evict to make space

➢ **Typical frame ranking criteria:** recency of use, frequency of use, number of processes reading it

➢ Typical optimization goal: Reduce total page I/O costs

❖ A few well-known policies:

- **Least Recently Used (LRU)**: Evict page that was used longest ago; **Most Recently Used (MRU)**: (Opposite of LRU)
- ML-based caching policies are "hot" nowadays!

Data Layouts and Access Patterns

❖ Recall that *data layouts* and *data access patterns* affect what data subset gets cached in higher level of memory hierarchy

- Recall matrix multiplication example and CPU caches

Key Principle: Optimizing layout of data file on disk based on data access pattern can help reduce I/O costs

❖ Applies to both magnetic hard disk and flash SSDs

- But especially critical for magnetic hard disks due to vast differences in latency of random vs sequential access!

Row-store vs Column-store Layouts

A common dichotomy when serializing 2-D structured data (relations, matrices, DataFrames) to file on disk

❖ Based on data access pattern of program, I/O costs with row- vs col-store can be orders of magnitude apart!

❖ Sometimes, it is beneficial to do a hybrid, especially for analytical RDBMSs and matrix/tensor processing systems

Key Principle: What data layout will yield lower I/O costs (row vs col vs tiled) depends on data access pattern of the program!

Example: Dask DataFrame

❖ Dask DF scales to disk-resident data via a row-store

❖ "Virtual" split: each split is a Pandas DF under the hood

❖ Dask API is a "wrapper" around Pandas API to scale ops to splits and put all results together

❖ If file is too large for DRAM, need manual *repartition*() to get physically smaller splits (< ~1GB)

Example: Modin's DataFrame

❖ Modin's DF aims to scale to disk-resident data via a tiled store

❖ Enables seamless scaling along both dimensions

❖ Easier use of multi-core parallelism

- Many in-memory RDBMSs had this, e.g., SAP HANA, Oracle TimesTen; ScaLAPACK had this for matrices

Scaling with Remote Reads

❖ Similar to scaling to local disk but not "local": Stage page reads from remote disk/disks over the network (e.g., from S3). *More restrictive* than scaling with local disk, since spilling is not possible or requires costly network I/Os. OK for a *one-shot* filescan access pattern

- Use DRAM to cache; dependency on replacement policies
- Can also use smaller local disk as cache (done in PA1)

Lecture 11: Scalable Data Access

Scaling Data Science Operations

Scalable data access is used in key representative examples of programs/operations that are ubiquitous in data science:

❖ **DB systems:** Relational select; Non-deduplicating (= not avoiding duplication) projects; Simple SQL aggregates; SQL GROUP BY aggregates

❖ **ML systems:** Matrix sum/norms;(Stochastic) Gradient Descent

Scaling to Disk: Non-deduplic. Project (SELECT C FROM R)

Straightforward ***filescan*** data access pattern

❖ Read one page at a time into DRAM; may need cache repl.

❖ Drop unneeded columns from tuples on the fly

❖ I/O cost: (read) + output # pages (write)

Big advantage for col-stores over row-stores for SQL analytics queries (projects, aggregates, etc.); popular in online analytical processing ("OLAP")

- Rationale for col-store RDBMS (e.g., Vertica) and Parquet

Scaling to Disk: Simple Aggregates (SELECT MAX(A) FROM R)

Again, straightforward ***filescan*** data access pattern

❖ Similar I/O behavior as non-deduplicating project

Scaling to Disk: Group By Aggregate (SELECT A, SUM(D) FROM R GROUP BY A)

❖ Operation is not straightforward due to the grouping request.

- Need to "collect" tuples in groups based on unique A values and apply aggregation function to each

❖ Typically done with a ***hash table*** maintained in DRAM

- Has 1 record per group and maintains "running information" for that group's aggregation function
- Built on the fly during filescan of R; holds the output in the end

❖ Note that the sum for each group is constructed ***incrementally***

***Q:** But what if hash table > DRAM size?*

Program will likely just crash! OS may keep swapping pages of hash table to/from disk; aka "thrashing"

***Q:** How to scale to large number of groups?*

Divide and conquer! Split up R based on values of A; HT for each split may fit in DRAM alone; Reduce running info size if possible

Scaling to Disk: Matrix Sum/Norms $||M||^2_2$

Again, straightforward filescan data access pattern

❖ Very similar to relational simple aggregate

❖ Running info in DRAM for sum of squares of cells

0 -> 5 -> 10 -> 15 -> 20 -> 30 -> 40

❖ I/O cost: 6 (read) + output # pages (write)

Scalable Matrix/Tensor Algebra

In general, tiled partitioning is more common for matrix/tensor ops

- More DRAM/cache-efficient implementations

❖ DRAM-to-disk scaling:

- For matrices: pbdR, SystemDS, Dask Arrays
- For n-d arrays: SciDB, Xarray

❖ CUDA for DRAM-GPU caches scaling of matrix/tensor ops

Numerical Optimization in ML

❖ Many regression and classification models in ML are formulated as a (constrained) minimization problem

- E.g., logistic and linear regression, linear Support Vector Machine, etc.
- "Empirical Risk Minimization" (ERM) approach "Loss" of predictions is computed over labeled examples

❖ Hyperplane-based models aka Generalized Linear Models (GLMs) use f() that is a scalar function of distances: $w^T x_i$

Batch Gradient Descent for ML

❖ In many cases, **loss function** l() is **convex**; so is L()

❖ Closed-form minimization typically infeasible ⇒iterative solution -> **Batch Gradient Descent:**

$$L(\mathbf{w}) = \sum_{i=1}^{n} l(y_i, f(\mathbf{w}, x_i))$$

❖ Iterative numerical procedure to find an optimal w

❖ Initialize w to some value w(0)

Compute **gradient**: $\nabla L(\mathbf{w}^{(k)}) = \sum_{i=1}^{n} \nabla l(y_i, f(\mathbf{w}^{(k)}, x_i))$

Descend along gradient: (Aka **Update Rule**) $\mathbf{w}^{(k+1)} \leftarrow \mathbf{w}^{(k)} - \eta \nabla L(\mathbf{w}^{(k)})$

❖ Repeat until we get close to w*, aka **convergence**

❖ Learning rate η is a hyper-parameter selected by user or "AutoML" tuning procedures

❖ Number of epochs of BGD also hyper-parameter. Designates the number of iterations (complete passes of training dataset).

❖ Very similar to algebraic SQL; vector addition

❖ **Input Split**: Shard table tuple-wise

❖ **Map()**: On tuple, compute per-example gradient; add these across ex in shard; emit partial sum with single dummy key

❖ **Reduce()**: Only one global dummy key, Iterator has partial gradients; just add all those to get full batch gradient

Data Access Pattern of BGD at Scale

❖ The data-intensive computation in BGD is the gradient

- In scalable ML, dataset D may not fit in DRAM
- Model w is typically small and DRAM-resident

***Q:** What SQL operation is this reminiscent of?*

❖ Gradient is like SQL SUM over vectors (one per example)

❖ At each epoch, 1 **filescan** over D to get gradient

❖ Update of w happens normally in DRAM

❖ Monitoring across epochs for convergence needed

❖ Loss function L() is also just a SUM in a similar manner

I/O Cost of Scalable BGD

❖ Straightforward **filescan** data access pattern for SUM

- Similar I/O behavior as non-dedup. project and simple SQL aggregates

Stochastic Gradient Descent for ML

❖ Two key cons of BGD:

- Often, too many epochs to reach optimal
- Each update of **w** needs full scan: costly I/Os

❖ Stochastic GD (SGD) mitigates both cons

❖ **Basic Idea**: Use a sample (**mini-batch**) of D to approximate gradient instead of "full batch" gradient

- Done *without replacement*
- Randomly reorder/shuffle D before every epoch
- Sequential pass: sequence of mini-batches

❖ Another big pro of SGD: works well for *non-convex* loss too, especially DL; BGD does not

❖ SGD often called the "workhorse" of modern ML/DL

I/O Cost of Scalable SGD

❖ I/O cost of random shuffle is non-trivial; need so-called "external merge sort" (skipped in this course)

- Typically amounts to 1 or 2 passes over file

❖ Mini-batch gradient computations: 1 filescan per epoch:

- As filescan proceeds, count # examples seen, accumulate per-example gradient for mini-batch
- Typical mini-batch sizes: 10s to 1000s
- Orders of magnitude more model updates than BGD!

❖ Total I/O cost per epoch: 1 shuffle cost + 1 filescan cost

- Often, shuffling only once upfront suffices

❖ Loss function L() computation is same as before (for BGD)

Lecture 12: Data Parallelism: Parallelism + Scalability

Basic Idea of Scalability: Split data file (virtually or physically) and *stage reads/writes* of its pages between disk and DRAM

***Q:** What is "data parallelism"?*

Data Parallelism: Partition large data file *physically* across nodes/workers; within worker: DRAM-based or disk-based

❖ The most common approach to marrying *parallelism* and *scalability* in data systems

❖ Generalization of SIMD and SPMD idea from parallel processors to large-scale data and multi-worker/multi-node setting ❖ Distributed-memory vs Distributed-disk

3 Paradigms of Multi-Node Parallelism

- Shared-Nothing, Shared-Disk, Shared-Memory Parallelism

Data parallelism is technically orthogonal to these 3 paradigms but most commonly paired with shared-nothing

Shared-Nothing Data Parallelism

❖ **Sharding**: Partitioning a data file across nodes.

❖ Part of a stage in data processing workflows called **Extract-Transform-Load (ETL)**

❖ ETL is an umbrella term for all kinds of processing done to the data file before it is ready for users to query, analyze, etc.

- Sharding, compression, file format conversions, etc.

Data Partitioning Strategies

❖ Row-wise/*horizontal* partitioning is most common (sharding)

❖ 3 common schemes (given k nodes):

- **Round-robin**: assign tuple i to node i MOD k
- **Hashing-based**: needs hash partitioning attribute(s)
- **Range-based**: needs ordinal partitioning attribute(s)

❖ **Tradeoffs**:

For Relational Algebra (RA) and SQL:

- Hashing-based most common in practice for RA/SQL
- Range-based often good for range predicates in RA/SQL

But all 3 are often OK for many ML workloads.

❖ **Replication** of partition across nodes (e.g., 3x) is common to enable "*fault tolerance*" and better parallel *runtime performance*

- Just like with disk-aware data layout on single-node, we can partition a large data file across workers in other ways too:

i.e. Columnar Partitioning, Hybrid/Tiled Partitioning

Cluster Architectures

***Q:** What is the protocol for cluster nodes to talk to each other?*

Manager-Worker Architecture

❖ 1 (or few) special node called **Manager** (aka "Server" or archaic "Master"); 1 or more Workers

❖ Manager tells workers what to do and when to talk to other nodes. Most common in data systems (e.g., Dask, Spark, par. RDBMS, etc.)

Peer-to-Peer Architecture

❖ No special manager, Workers talk to each other directly; E.g., Horovod

❖ Aka Decentralized (vs Centralized)

Bulk Synchronous Parallelism (BSP)

❖ Most common protocol of data parallelism in data systems (e.g., in parallel RDBMSs, Hadoop, Spark)

❖ Shared-nothing sharding + manager-worker architecture

1. Sharded data file on workers
2. Client gives program to manager (e.g., SQL query, ML training)
3. Manager divides first piece of work among workers
4. Workers work independently on self's data partition (cross-talk can happen if Manager asks)
5. Worker sends partial results to Manager
6. Manager waits till all k done
7. Go to step 3 for next piece

Speedup Analysis/Limits of of BSP

***Q:** What is the speedup yielded by BSP?*

Speedup = *Completion time given only 1 worker / Completion time given k (>1) workers*

Cluster overhead factors that hurt speedup:

❖ **Per-worker**: startup cost; tear-down cost

❖ **On manager**: dividing up the work; collecting/unifying partial partial results from workers

❖ **Communication costs**: talk between manager-worker and across workers (when asked by manager)

❖ Barrier synchronization suffers from "**stragglers**" (workers that fall behind) due to skews in shard sizes and/or worker capacities

Distributed Filesystems

Recall definition of file; distributed file generalizes it to a cluster of networked disks and OSs. **Distributed filesystem (DFS)** is a cluster-resident filesystem to manage distributed files

❖ A layer of *abstraction* on top of local filesystems

❖ Nodes manage local data as if they are local files

❖ *Illusion of a one global file*: DFS APIs let nodes access data sitting on other nodes

❖ 2 main variants: Remote DFS vs In-Situ (on-site) DFS

- **Remote DFS**: Files reside elsewhere and read/written on demand by workers
- **In-Situ DFS**: Files resides on cluster where workers exist

Network Filesystem (NFS)

An old remote DFS (c. 1980s) with simple client-server architecture for *replicating* files over the network

❖ *Main pro*: simplicity of setup and usage

❖ *But many cons*: Not scalable to very large files; Full data replication; High contention for concurrent reads/writes; Single-point of failure

Hadoop Distributed File System (HDFS)

❖ Most popular in-situ DFS (c. late 2000s); part of Hadoop; open source spinoff of Google File system (GFS)

❖ *Highly scalable;* scales to 10s of 1000s of nodes, PB files

❖ Designed for clusters of cheap commodity nodes

❖ Parallel reads/writes of sharded data "blocks"

❖ Replication of blocks to improve fault tolerance

❖ Cons: Read-only + batch- append (no fine-grained updates/writes)

❖ NameNode's roster: Maps data blocks to DataNodes/IPs

❖ A distributed file on HDFS is just a directory (!) with individual filenames for each data block and metadata files

❖ ***HDFS has configurable parameters:***

Data block size: Splitting data into chunks, 128 MB

Replication factor: Ensure data availability, 3x

Data-Parallel Dataflow/Workflow

❖ **Data-Parallel Dataflow**: A dataflow graph with ops wherein each operation is executed in a data-parallel manner

❖ **Data-Parallel Workflow**: A generalization; each vertex a whole task/process that is run in a data-parallel manner

Note: In parallel environments like parallel RDBMSs and Spark: Each of these extended relational ops have scalable data-parallel implementations.All input tables are sharded

---

Lecture 13: Data-Parallel Data Science Operations

Data-Parallel Non-deduplic. Project

We focus on Bulk Synchronous Parallelism (BSP) data-parallelism. **Basic Idea:** Manager splits work -> node-local work -> manager unifies results

1. After ETL, sharded large input file sits inside cluster's disks
2. When query/program given, Manager broadcasts it as such
3. Each worker does node-local Non-dedup Project as explained before and writes local output to local file
4. Manager reports union of local files as global output file

Data-Parallel Simple Aggregates

1 and 2 same as above.

3. Each worker does node-local simple **partial aggregate** as explained before and sends it to the Manager for unification
4. Manager unifies partial results based on operation semantics

***Q:** Are all SQL aggregates easy to split up on sharded data?*

❖ Based on how easy it is to split up on shards, SQL aggs (aka descriptive stats) are categorized into 2-3 types:

❖ **Distributive Aggs**: A shard sends only 1 datum to manager

- MIN, MAX, COUNT, SUM

❖ **Algebraic Aggs**: A shard sends O(1) size stats to manager

- AVG (send SUM, COUNT separately); VARIANCE and STDEV (send SUM, SUM of squares, COUNT); etc.

❖ **Holistic Aggs**: Just O(1) size stats not enough in general; may need larger intermediate stats

- MEDIAN, MODE, PERCENTILES, etc.

Data-Parallel Group By Aggregate

Similar to data-parallel simple agg; Workers send **partial hash table** to manager based on local shards. Manager collects and unifies local hash tables into global output. Network I/O cost depends on data stats (domain size of A)

Data Access Pattern of Scalable SGD

An SGD epoch is similar to SQL aggs but also different:

❖ More complex agg. state (running info): model param. $\mathbf{W}^{(t)}$

❖ Multiple mini-batch updates to model param. within a pass

❖ Sequential dependency across mini-batches in a pass

❖ Keep track of model parameters across epochs

❖ Not an algebraic aggregate; *hard to parallelize*!

❖ **Not** commutative: different random shuffle orders give different results (very unlike relational ops)

❖ (Optional) New random shuffling before each epoch

Execution Optimization Tradeoffs

Some common optimizations in data-parallel systems:

❖ **Replication**: Put a shard on >1 worker; more parallelism possible for execution

❖ **Caching**: Store as much data as possible on worker DRAM and/or disk

❖ **Asynchrony**: Less common in DB systems; more common in ML systems (e.g., ParameterServer)

❖ **Approximation**: Carefully exploit data subsampling

Using ML for data placement, caching, tiered storage across memory hierarchy is now a hot topic in "ML for systems" world

Hybrid Parallelism

Task- vs Data-Parallelism have pros and cons:

❖ Task-parallelism wastes memory/storage due to replication; remote reads waste network; but easy to implement

❖ Data-parallelism is painful to implement at op level; but scales w/o wasting memory/storage; more network costs

***Q:** Is it possible to get the best of both these worlds?*

❖ Yes, often we can run task-parallelism on sharded data!

❖ Examples: Different SQL queries or different ML training routines run on top of same sharded data setup

- Aka "**Multi-Query Execution**" in the DB world

Hybrid of Task and Data Parallelism

***Q:** Can we go faster if we hybridize task and data par?*

Most scalable data systems today support only full task-par. (e.g., Dask) or full data-par. (e.g., RDBMS); hybrid software complexity is high. Some RDBMSs do internally exploit hybrid-par. for relational dataflows. Spark is beginning to support task-par. too

A key example from 2021 research:

Cerebro for parallel DL model selection on clusters. First known form of "Bulk *Asynchronous* Parallelism". Resource-optimal when compute, memory/storage, and network considered holistically.

Lecture 14: Dataflow Systems

Parallel RDBMSs

❖ Parallel RDBMSs have been highly successful and widely used

❖ Typically shared-nothing data parallelism

❖ Optimized **runtime performance** + enterprise-grade features:

- ANSI SQL & more
- Business Intelligence (BI) dashboards/APIs
- Transaction management; crash recovery
- Indexes, auto-tuning, etc.

Beyond RDBMSs: A Brief History

❖ DB folks got blindsided by the rise of Web/Internet giants

4 new concerns of Web giants vs RDBMSs built for enterprises:

❖ **Developability**: Custom data models and computations hard to program on SQL/RDBMSs; need for simpler APIs

❖ **Fault Tolerance**: Need to scale to 1000s of machines; need for graceful handling of worker failure

❖ **Elasticity**: Need to be able to easily upsize or downsize cluster size based on workload

❖ **Cost**: Commercial RDBMSs licenses too costly; hired own software engineers to build custom new systems

*A new breed of parallel data systems called **Dataflow Systems** jolted the DB industry from being overconfident and complacent!*

What is MapReduce?

❖ A programming model for parallel programs on **sharded data + distributed system architecture**

❖ **Map** & **Reduce** are functional programming languages terms Software/data/ML engineers implement logic of Map, Reduce

❖ System handles data distribution, parallelization, fault tolerance, etc. *under the hood*

❖ Created by Google to solve "simple" data workload: index, store, and search the Web!

❖ Google's engineers started with MySQL! Abandoned it due to reasons listed earlier (developability, fault tolerance, elasticity)

***Goal**: High-level functional ops to simplify data-intensive programs*

❖ **Key Benefits:**

- Map() and Reduce() are highly general; work with any data types/structures; great for ETL, text/multimedia
- Native scalability, large cluster parallelism
- System handles fault tolerance automatically
- Decent FOSS stacks (Hadoop and later, Spark)

❖ **Catch**: Users must learn "art" of casting program as MapReduce

- Map operates record-wise; Reduce aggregates globally
- But MR libraries now available in many PLs: C/C++, Java, Python, R, Scala, etc.

Abstract Semantics of MapReduce

Map(): Process one "record" at a time independently

- A record can physically batch multiple data examples/tuples
- Dependencies across Mappers not allowed
- Emit one or more key-value pairs as output(s)
- Data types of input vs. output can be different

Reduce(): Gather all Map outputs across workers sharing same key into an Iterator (list)

- Apply aggregation function on Iterator to get final output(s)

Input Split:

- Physical-level shard to batch many records to one file "block" (HDFS default: 128MB / configurable)
- User/application can create custom Input Splits

Emulate MapReduce in SQL?

***Q:** How would you do the word counting in RDBMS / in SQL?*

❖ First step: **Transform** text docs into relations and load: Part of the ETL stage; Suppose we pre-divide each doc into words w/ schema: **DocWords** (DocName, Word)

❖ Second step: a single, simple SQL query!

SELECT Word, COUNT (*) FROM DocWords GROUP BY Word [ORDER BY Word] Parallelism, scaling, etc. done by RDBMS under the hood

More MR Examples: Select Operation

- **Input Split**: Shard table tuple-wise
- **Map()**: On tuple, apply selection condition; if satisfies, emit key-value (KV) pair with dummy key, entire tuple as value
- **Reduce()**: Not needed! No cross-shard aggregation here

*These kinds of MR jobs are called "**Map-only**" jobs*

More MR Examples: Simple Agg.

Suppose it is algebraic aggregate (SUM, AVG, MAX, etc.)

- Input Split: Shard table tuple-wise

- Map(): On agg. attribute, compute incremental stats; emit pair with single global dummy key and incremental stats as value

- Reduce(): Since only one global dummy key, Iterator has all sufficient stats to unify into global agg.

More MR Examples: GROUP BY Agg.

Assume it is algebraic aggregate (SUM, AVG, MAX, etc.)

- Input Split: Shard table tuple-wise

- Map(): On agg. attribute, compute incremental stats; emit pair with grouping attribute as key and stats as value

- Reduce(): Iterator has all sufficient stats for a single group; unify those to get result for that group; Different reducers will output different groups' results

More MR Examples: Matrix Norm

Assume it is algebraic aggregate (Lp,q norm); Very similar to simple SQL aggregates

- Input Split: Shard table tuple-wise

- Map(): On agg. attribute, compute incremental stats; emit pair with single global dummy key and stats as value

- Reduce(): Since only one global dummy key, Iterator has all sufficient stats to unify into global aggregate.

What is Hadoop?
Hadoop is a FOSS system implementation with MapReduce as programming model, and HDFS as filesystem. MR user API; input splits, data distribution, shuffling, and fault tolerances handled by Hadoop under the hood; Exploded in popularity in 2010s: 100s of papers, 10s of products; A "revolution" in scalable+parallel data processing that took the DB world by surprise; But nowadays Hadoop largely supplanted by Spark
NB ("Nota bene" = Mark well): Do not confuse MapReduce for Hadoop or vice versa!

Lecture 15: Spark and Dataflow Programming
Apache Spark Dataflow programming model (subsumes most of Relational Algebra; MR)
- Inspired by Python Pandas style of chaining functions
- Unified storage of relations, text, etc.; custom programs
- System implementation (re)designed from scratch
❖ Tons of sponsors, gazillion bucks, unbelievable hype!
❖ **Key idea vs Hadoop**: exploit distributed memory to cache data
❖ **Key novelty vs Hadoop**: lineage-based fault tolerance
❖ Open-sourced to Apache; commercialized as Databricks
A simple extension to MapReduce called Resilient Distributed Datasets (RDDs), which just adds efficient data sharing primitives, greatly increases its generality. RDDs are best suited for batch applications that apply the same operation to all elements of a dataset.
Resilient Distributed Datasets
❖ RDD has been the primary user-facing API in Spark since its inception. At the core an RDD is an immutable distributed collection of elements of your data,
- partitioned across nodes in your cluster
- that can be operated in parallel with a low-level API that offers transformations and actions.
❖ Good for dataset low-level transformation, actions and control.
❖ Good for unstructured data.
❖ Good for functional programming data manipulation.
❖ Not recommended for imposing a schema on your data.
❖ Lacks some optimization and performance benefits
Spark DF API and SparkSQL
Databricks recommends SparkSQL/DataFrame API; avoid RDD API unless really needed! **Key Reason**: Automatic query optimization becomes more feasible
Query Optimization in Spark
❖ Common automatic query optimizations (from RDBMS world) are now performed in Spark's Catalyst optimizer:
❖ **Projection pushdown:** Drop unneeded columns early on
❖ **Selection pushdown:** Apply predicates close to base tables
❖ **Join order optimization:** Not all joins are equally costly
❖ Fusing of aggregates …
Comparing Spark's APIs A rough comparison of RDD, DataFrames and Koalas (a Databricks, pandas-like module)

	RDD	DataFrame	Koalas
Abstraction Level	Low	High	High
Named Columns	No	Yes	Yes
Support for Query Optimization	No	Yes	Yes
Programming Mode	map-reduce	Dataflow, SQL	Pandas-like
Best suited for	Unstructured data Low-level ops Folks who like func. PLs and MapReduce	Structured data High-level ops Folks who know SQL, Python, R	Structured data Lower barrier to entry. For folks who only know Pandas or Dask

Data "Lakehouse": A New Paradigm
Data Lake: *Loose coupling* of data file format and data/query processing stack (vs RDBMS's tight coupling); many frontends
❖ Unified storage in a single, low-cost data store
❖ Data Management tools providing features
❖ Atomicity, Consistency, Isolation, and Durability: Provide transactional support for data consistency
❖ Standardized formats: Apache Iceberg, Delta Lake (OSS, integrated w/ Databricks)
❖ Scalable infrastructure
❖ Data governance: Access control, auditing, metadata management, lineage
❖ Metadata and cataloging enable self-service Analytics Data Lakehouses

---

Lecture 16: Model Building Systems
Building Stage of ML Lifecycle
Perform **model selection**, i.e., convert prepared ML-ready data to **prediction function(s)** and/or other analytics outputs
What makes model building challenging/time-consuming?
❖ **Heterogeneity** of data sources/formats/types
❖ **Configuration complexity** of ML models
❖ Large **scale** of data
❖ **Long training runtimes** of some models
❖ **Pareto optimization on criteria** for application
❖ **Evolution** of data-generating process/application

*Data scientist / ML engineer must steer 3 key activities that invoke ML **training** and **inference** as sub-routines:*
1. Feature Engineering (FE): Appropriate signals representation for domain of prediction function.
2. Algorithm/Architecture Selection (AS): Choice of prediction functions class (incl. artificial Neural Net architecture) to use.
3. Hyper-parameter Tuning (HT): Model improvement (accuracy, etc.) by configuring ML "knobs".
Model Selection Process
❖ Model selection is usually an *iterative exploratory process* with human making decisions on FE, AS, and/or HT
❖ Increasingly, automation of some or all parts possible: **AutoML**
❖ Decisions on FE, AS, HT guided by many constraints/metrics: prediction accuracy, data/feature types, interpretability, tool availability, scalability, runtimes, fairness, legal issues, etc.
❖ Decisions are typically application-specific and dataset-specific;
recall Pareto surfaces and tradeoffs
Feature Engineering
Converting prepared data into a *feature vector representation* for ML training and inference.
1. Recoding and value conversions Common on relational/tabular data; Typically needs some global column stats + code to reconvert each tuple (example's feature values).
2. Joins and Aggregates Common on relational/tabular data; Most real-world relational datasets are multi-table; require key-foreign key joins, aggregation-and-key-key-joins, etc.
3. Feature Interactions Sometimes used on relational/tabular data, especially for high-bias models like GLMs; Pairwise is common; ternary is not unheard of; No global stats, just a tuple-level function; NB: Popularity of this has reduced due to kernel Support Vector
Machines; but so-called "factorization machines" still need this
4. Feature Selection Sometimes used on relational/tabular data; **Basic Idea**: Instead of using whole feature set, use a subset. Formulated as a discrete optimization problem: NP-Hard in number of features in general. Many heuristics exist in ML/data mining; typically rely on some information theoretic criteria; Typically scaled as "outer loops" over training/inference. Some ML users also prefer human-in-the-loop approach
5. Dimensionality reduction Often used on relational/structured/ tabular data. **Basic Idea**: Transforms features to a different latent space **Examples**: Principal Component Analysis (PCA), Singular Value Decomposition (SVD), Linear Discriminant Analysis (LDA), Matrix factorization
❖ Feature selection *preserves* semantics of each feature but dimensionality reduction typically does not—combines features in "nonsensical" ways. Scaling this is non-trivial! Similar to scaling individual ML training algorithms
6. Temporal Feature Extraction Many relational/tabular data have time/date: Per-example reconversion to extract numerics/ categoricals, Sometimes global stats needed to calibrate time, Complex temporal features studied in time series mining
7. Textual Feature Extraction Many relational/tabular data have text columns; in NLP, whole example is often just text, Most classifiers cannot process text/strings directly, Extracting numerics from text studied in text mining.
❖ **Knowledge Base-based:** Domain-specific knowledge bases like entity dictionaries (e.g., celebrity or chemical names) help extract domain-specific features
❖ **Embedding-based:**
- Numeric vector for a text token; popular in NLP
- Offline training of function from string to numeric vector in self-supervised way on large text corpus (e.g., Wikipedia); embedding dimensionality is a hyper-parameter
- Pre-trained word embeddings (Word2Vec and GloVe) and sentence embeddings (Doc2Vec) available off-the-shelf; to scale, just use a tuple-level conversion function
8. Learned Feature Extraction in DL A big win of Deep Learning (DL) is no manual feature engineering on *unstructured* data
❖ **NB**: DL is *not* common on structured/tabular data!
DL is very *versatile*: almost *any data type* as input and/or output:
- Convolutional NNs (CNNs) over image tensors
- Recurrent NNs (RNNs) and Transformers over text
- Graph NNs (GNNs) over graph-structured data
❖ Neural architecture specifies how to extract and transform features internally with weights that are learned
❖ **Software 2.0**: Buzzword for such "learned feature extraction" programs vs old hand-crafted feature engineering

---

Lecture 17: Model Building System
Hyper-Parameter Tuning
❖ **Hyper-parameters**: Knobs for an ML model or training algorithm to control *bias-variance* tradeoff in a dataset-specific manner to make learning effective
Examples:
- GLMs: L1 or L2 regularizer to constrain weights
- All gradient methods: learning rate
- Mini-batch Stochastic Gradient Descent: batch size
❖ HT is an "outer loop" around training/inference
❖ Most common approach: **grid search**; pick set of values for each hyper-parameter and take cartesian product
❖ Also common: **random search** to subsample from grid
❖ Complex AutoML heuristics exist too for HT, e.g., HyperOpt
Algorithm Selection
❖ Not much to say; ML user typically picks models/algorithms ab initio in **"classical" ML** (non-DL)
❖ Best practice: first train more simple models (e.g., regression-based ones) as baselines; then, try more complex models (XGBoost)
❖ **Ensembles**: Build diverse models and aggregate predictions
Architecture Selection in DL
❖ More critical in DL; neural architecture is **inductive bias** in classical ML parlance; controls feature learning and bias-variance tradeoff
❖ Some applications: Off-the-shelf, pre-trained DL models for "transfer learning"
- Very popular at HuggingFace prior to the advent of AI models
❖ Other applications: Swap pain of hand-crafted feature engineering for pain of neural architecture engineering!
Automated Model Selection / AutoML
***Q:** Can we automate the whole model selection process?*
It depends. HT and most of FE already automated mostly in practice; (neural) AS is often application-dictated. AutoML tools/systems now aim to reduce data scientist's work; or even replace them...
❖ **Pros**: Ease of use; lower human cost; easier to audit; improves ML accessibility
❖ **Cons**: Higher resource cost; less user control; may waste domain knowledge
❖ Pareto-optima; hybrids possible; The Data Sourcing stage is still very hard to automate!
Major ML Model Families/Types
Generalized Linear Models (GLMs); from statistics
Bayesian Networks; inspired by causal reasoning
Decision Tree-based: CART, Random Forest, Gradient-Boosted Trees (GBT), etc.; inspired by symbolic logic
Support Vector Machines (SVMs); inspired by psychology
Artificial Neural Networks (ANNs): Multi-Layer Perceptrons (MLPs), Convolutional NNs (CNNs), Recurrent NNs (RNNs), Transformers, etc.; inspired by brain neuroscience
Unsupervised: Clustering (e.g., K-Means), Matrix Factorization, Latent Dirichlet Allocation (LDA), etc.
Scalable ML Training Systems
❖ Scaling ML training is involved and model type-dependent
❖ Orthogonal dimensions of categorization:
1. Scalability: In-memory libraries vs Scalable ML system (works on larger-than-memory datasets)
2. Target Workloads: General ML library vs. Decision tree-oriented vs. Deep learning, etc.
3. Implementation Reuse: Layered on top of scalable data system vs. Custom from-scratch framework
Scalable ML Inference A trained/learned ML model is just a prediction function: f: D_X -> D_Y
***Q:** Given large dataset of examples, how to scale inference?*
❖ Assumption 1: An example fits entirely in DRAM
❖ Assumption 2: f fits entirely in DRAM
❖ If both hold, trivial access pattern: single filescan, apply per-tuple function f, write output. How to do this with MapReduce
❖ If either fails, access pattern becomes more complex, and dependent on breaking up internals of f to stage access to data for partial computations

Assume you parallelize a data-intensive program across multiple cores on a single machine. 30% of the program is inherently sequential, while the rest can yield perfect linear speedup for any number of cores.
The single-core runtime of the program is 90min.

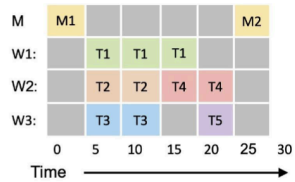
1. What is the runtime (in minutes) of parallel 9-core execution?

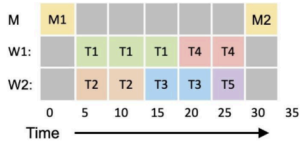
Answer: The time consumed in sequential execution: 90' * (30/100) = 27'.
Remainder to be parallelized: 90' – 27' = 63'.
Over 9 cores, perfect linear speedup means execution will take 63' / 9 = 7'.
Total execution time: 27' + 7' = 34'.

2. What is the overall speedup factor for the program? If needed, use up to 1 decimal digit to approximate your answer.

Answer: 90' / 34' = 2.6

Consider this workload with 5 tasks T1 – T5 being run in a task-parallel manner on a system with 3 workers and a manager node. Task T5 depends on T1. When a task starts, it runs uninterrupted until it completes on the same worker. With these rules, it takes 30 time units for the workload to complete on 3 workers.
What is the lowest possible task-parallel completion time of this workload given only 2 workers W1 and W2?





Answer: With 2 workers, any 2 the T1-T4 tasks can start running in parallel. Let us assume first we have T1 run on W1 and T2 run on W2.
T2 will be done faster than T1. T3 or T4 can then run on W2.
Let us assume T3 follows T2. This means T4 or T5 can run after T1 is complete on W1.
However, to achieve lowest completion time, T4 must follow T1 on W1, and T5 must start on W2 after T3 is complete. (Otherwise, T4 will be starting on W2 when T5 is done on W1.)
The workload will complete at the lowest of 35 time units.

EXERCISE: Compute Cost

Use case of amplifying the accuracy of a linear model with a multi-step workflow.
You are given a large training dataset with schema D (Y, X1, X2, ..., Xd). The target Y and all features are numeric and stored as float64.
The number of examples is n. The dataset is laid out in row store format on disk (no compression) on your single machine. No batching of file scans or caching of data is performed.
Step A. You build linear models using BGD. You try 10 different learning rates and train all models for 3 epochs each.
Step B. You read the data and append quadratic (second order) pairwise feature interactions. You write out the output file to disk for future use. All features are still stored as float64.
Given: (n = 500 million, d = 2)

First, let us process information: Each row has d+1 numbers ⇒ (d+1)*64 bits ⇒ (d+1)*8 bytes
For the given d=2 ⇒ The dataset has d+1 = 2+1 = 3 columns, and each row is 3*8=24 bytes
n = 500,000,000 or n = 5e8
A indicates there are 10 learning rates, so there is a total of 10 different models.
A also indicates 3 epochs for each model. This is BGD, so each epoch is a full dataset scan.
B specifies quadratic pairwise feature interactions, so we need to consider as a new feature each one of all possible combinations X1X1, X1X2, ..., X1XD, X2X3, ..., X2XD, ..., XDXD.

1. What is the size in GB of the file D on disk?

Answer: The size is (row size)*(# rows) = 24 * 5e8 = 120e8 = 12e9 bytes = 12 GB

2. What is the total I/O cost in GB of step A for reading data files?

Answer: For Step A, we need to read the dataset 10*3 = 30 times ⇒ 12 GB * 30 = 360 GB.

3. What is the size in GB of the file written out in step B?

Answer: For d=2: Interactions are among the pair: (x1, x2). This means we have to consider the interaction terms x1x1, x1x2, x2x2. We need a separate column for each one of these interactions; therefore 3 more columns are needed. The updated dataset will have 6 columns.
Each row will now consume 6*8 = 48 bytes.
Total output will be (row size)*(# rows) = 48 * 5e8 = 240e8 bytes = 24 GB.

4. What is the degree of parallelism of this entire 2-step workflow if you were to run it in a task-parallel manner?

Answer: A different worker can be used to run all the epochs for each learning rate. So for the 10 learning rates in step A we can run the step on 10 workers simultaneously.
Step B can be performed independently from A, hence an additional worker can be used for it.
Therefore, the total number of 10+1 = 11 workers that can be used simultaneously is the degree of parallelism for this use case.

EXERCISE: Workflow

Assume you need to run a classification workflow with 3 alternative feature engineering (FE) approaches. On each feature set, you apply a Random Forest (RF) model with 2 different hyperparameter combinations for each FE approach.

1. How many workers are necessary to achieve lowest-possible completion time with task parallelism no matter the tasks' runtimes?

Answer: Lowest completion time can be achieved when we take maximum advantage of parallelism. Therefore, the question just asks what this workflow's degree of parallelism is.
To compute this, we need to calculate the number of different workflow variations that can be each run independently on a different worker. The number of variations are the possible combinations across all alternative FE approaches and RF models, which is 3*2 = 6.
Therefore, one needs 3*2 = 6 workers.

2. Assume the task runtimes are known as follows:
-- 30, 35, and 40 units for the FE approaches.
-- For the 2 different hyperparameter combinations in each feature set: 20 and 30 units to build the RF models.
What is the upper bound on the lowest-possible completion time for this workflow?

Answer: We need an upper bound on the cost of the longest path. The longest path would be at most 40 units for the FE approach, and at most 30 units for the RF models.
So the longest path upper bound will be 40+30 = 70 time units.

3. What is the speedup when hitting the upper bound vs. using just 1 worker with no idle times for your whole workflow? If needed, use up to 1 decimal digit to approximate your answer.

Answer: For the speedup we need the ratio $s = t_1 / t_n$,
where t_1=[Completion time given 1 worker] and t_n=[Completion time given n workers].
The completion time t_n will be the slowest possible combination of FE approaches and RF models. The answer about the longest path upper bound in question 2 tells us exactly this completion time value that is t_n = 70.
For the total 1-worker time we sum up the times needed:
[time for all FE approaches] + [time to run both RF models for each of the 3 FE approaches] = [30+35+40] + [3*(20+30)] = 105 + 150 = 255 = t_1
In conclusion, the requested speedup is 255/70 = 3.6 (1 decimal digit approximation.)

EXERCISE: Compute Cost

You are given a large float64 matrix M of dimensions 25 million x 10 million stored in tiled layout without compression. The tiles are squares of dimensions 2000 x 2000. They stride along both dimensions by 2000 cells starting from top left. Assume a tile's data fits exactly on one page on disk.
What is the rough minimum disk I/O cost (in pages) of following linear algebra computation?
Summation of M[1 : 5,000,000][2,000,001 : 4,000,000]

Notes: M[i:j][p:q] means the system reads only rows i to j and columns p to q (ends included).
Row/column indices start from 1.
Exclude output write costs. Assume the DRAM cache is initially empty.

Answer: The requested cost is the cost to read the number of tiles that are involved in the specified computation. For this computation, only some of the tiles are needed. Observe that the indices of the selected rows and columns align conveniently with tile boundaries.
Specifically, with respect to
a) rows: The first $5*10^6$ rows participate, which correspond to $5*10^6 / 2*10^3$ = 2500 tiles.
b) columns: Of the $4*10^6$ total rows, only $4*10^6 - 2*10^6 = 2*10^6$ columns participate. These correspond to $2*10^6 / 2*10^3$ = 1000 tiles.
Therefore, a total of 2500*1000 = 2,500,000 tiles are involved in the computation.
Since a tile's data fits exactly on one page on disk, the rough minimum disk I/O in pages is 2500000.

Assume you are given a large dataset file for ML training that is of size 120 GB. What is the lowest possible I/O cost (in GB) of each of the following feature engineering operations? Ignore final output write costs and any potential gains due to caching.

1. Quadratic (order2) feature interactions.

Answer: 120 GB (only a single file scan is needed).

2. Binning a numeric feature into 10 given intervals.

Answer: 120 GB (only a single file scan is needed).

3. One-hot encoding of a categorical feature (assume feature's domain has only 5000 unique values).

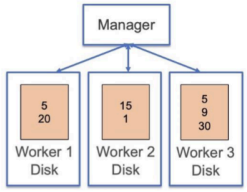
Answer: 120 GB (only a single file scan is needed).

4. Whitening a numeric feature by normalizing the values.

Answer: 240 GB (one file scan is needed to compute mean and stdev; a second file scan is needed to whiten feature values).

EXERCISE: Map-Reduce

You have the dataset
A = [1, 5, 5, 9, 15, 20, 30]
and you want to program map-reduce code to compute the average of the numbers in A on a 3-worker system.
For the reduce phase, your code for the Manager node only adds up the partial results of the Worker nodes to produce the desired outcome.



Given the above information, and that
-- the data are sharded randomly across the workers drives as shown in the figure, and
-- the Manager node passes as input to each Worker node the population size of A
you want to use the correct algebraic implementation in your code to calculate the average of A.
Answer the following questions. If needed, use 4 decimal digits to approximate your answer.

1. What is the average of A?

Answer: A is a set with population size 7. The average value is
(1+5+5+9+15+20+30) / 7 = 12.1428

2. What is the partial result that Worker 1 should return to the Manager node?

Answer: Worker 1 should return the weighted average of its data shard:
2/7 * (5+20)/2 = 3.5714

3. What is the partial result that Worker 2 should return to the Manager node?

Answer: Worker 2 should return the weighted average of its data shard:
2/7 * (15+1)/2 = 2.2857

4. What is the partial result that Worker 3 should return to the Manager node?

Answer: Worker 3 should return the weighted average of its data shard:
3/7 * (5+9+30)/3 = 6.2857

Note 1: Observe: The sum of the partial results correctly gives the sought average of the set A.
Note 2: If it were specified that in the reduce phase the Manager node can perform more elaborate computations than just add partial results, an alternative solution would be for each Worker to return just the subset average. The Manager node should then weigh each Worker's partial result by the ratio of the Worker's subset population over the A population.